

Deploying DepthWizard to Hugging Face Spaces

Free tier • CPU Basic (2 vCPU, 16 GB RAM) • No credit card • Docker SDK

Why Hugging Face Spaces

The free CPU Basic tier gives 2 vCPU and 16 GB RAM. DepthWizard needs roughly 4 GB, so this fits comfortably. Render's free tier caps at 1 GB and will not hold SAM. Vercel and Netlify are serverless with short function timeouts and cannot run a 65-second inference request.

Two things to know. Spaces sleep after roughly 48 hours of inactivity, so open the URL once before any demo to warm it. And a free Space is public, so your code will be visible — fine at submission, but make it a deliberate choice.

Before you begin

- A device with unrestricted internet (huggingface.co must be reachable)
- Your repository with everything merged into **main** and pushed to GitHub
- Git installed; the Hugging Face CLI is optional
- 30 to 45 minutes for the first deployment

Confirm you are on main before you start. The Space builds whatever main contains.

```
git checkout main
git status          # must be clean
git log --oneline -3
```

Step 1 — Create the Hugging Face account

- Go to **huggingface.co/join** and sign up. Free, no card.
- Verify your email.
- Go to **Settings > Access Tokens** and create a token with **write** permission. Copy it — you need it to push.

Step 2 — Create the Space

Field	Value
Owner	your username
Space name	depthwizard
License	mit (or apache-2.0)
SDK	Docker — not Gradio, not Streamlit
Docker template	Blank
Hardware	CPU basic • 2 vCPU • 16 GB • FREE
Visibility	Public (private Spaces need a paid plan)

Click **Create Space**. You now have an empty git repository at `huggingface.co/spaces/USERNAME/depthwizard`

Step 3 — Verify your three required files

These should already exist in the repository. Check each one rather than recreating it.

3a. README.md — YAML block must be the very first characters

Spaces reads this frontmatter to configure the Space. If it is missing, or anything precedes it, the build fails or the Space is misconfigured.

```
---
title: DepthWizard
emoji: [pick any single emoji character here]
colorFrom: blue
colorTo: indigo
sdk: docker
app_port: 7860
pinned: false
license: mit
---
```

```
# DepthWizard - SIH 26175
```

Single-view height estimation and 3D flythrough from one optical satellite image. Shadow photogrammetry with LOD1 building extrusion.

app port: 7860 is not optional. Spaces routes traffic to port 7860. Your FastAPI app must listen there, not on 8000, 8001 or 8002.

3b. Dockerfile

Verify it matches this shape. Key points: non-root user, HF cache under the user home, port 7860, and a health check.

```
FROM python:3.12-slim

RUN apt-get update && apt-get install -y --no-install-recommends \
    libexpat1 libgl1 libglib2.0-0 curl \
    && rm -rf /var/lib/apt/lists/*

RUN useradd -m -u 1000 user
USER user
ENV HOME=/home/user \
    PATH=/home/user/.local/bin:$PATH \
    HF_HOME=/home/user/.cache/huggingface \
    TORCH_HOME=/home/user/.cache/torch \
    PYTHONUNBUFFERED=1

WORKDIR $HOME/app

COPY --chown=user requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

COPY --chown=user . .

EXPOSE 7860

HEALTHCHECK --interval=30s --timeout=10s --start-period=180s --retries=3 \
    CMD curl -f http://localhost:7860/health || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "7860"]
```

Set --start-period generously. Model download plus load can take several minutes on first boot, and a short start period will mark the container unhealthy before it has finished starting.

3c. requirements.txt

- Every version pinned.
- No Windows-only packages: remove pywin32, pywinpty, and anything similar.
- Use opencv-python-headless, not opencv-python — the headless build needs no display server and keeps the image smaller.
- For the CPU-only PyTorch wheel, add --extra-index-url https://download.pytorch.org/whl/cpu as the first line. The default wheel pulls CUDA libraries and adds gigabytes.

Step 4 — Model weights

SAM and Depth Anything weights are roughly 1 GB combined and are gitignored, so they are not in the repository.

Important: the Space itself *can* reach huggingface.co, even though the development laptop cannot. So let the container download them on first boot. download_models.py already does this. Weights land in \$HF_HOME, which persists between restarts but not between rebuilds.

- Model loading must run in a background thread so the server responds immediately — this is already implemented alongside the /health endpoint.
- First boot takes several minutes. That is expected.
- If the download fails, the log must say so loudly and the UI must show it — never fall back silently.

Fallback if downloads prove unreliable: commit the weights via Git LFS. Spaces repositories support large files.

```
git lfs install
git lfs track "*.pth" "*.safetensors"
git add .gitattributes models/
git commit -m "Add model weights via LFS"
```

Step 5 — Demo scenes (do not skip)

The pipeline takes about 65 seconds per scene at the default SAM 512/12 setting. A judge who must upload a file and wait that long may never see your output.

- Precomputed results for the demo scenes should be committed under `static/demo/` — JSON plus compressed JPEG textures, not the source GeoTIFFs. They are small and must not be gitignored.
- **Antakya crop2** — best result: 22 of 23 measured, $R = 0.681$
- **Antakya crop1** — borderline tier, 12 buildings, azimuth agreement 5.4°
- **Kathmandu crop** — the negative case, showing the gates declining metric output at 74° sun elevation
- Landing page must present these as obvious buttons. No timed auto-redirect — it races against the user's own click.
- Label precomputed results clearly as such in the UI, so the two-second render is never mistaken for pipeline speed.

Including the Kathmandu failure is deliberate. It demonstrates the quality gates working, which almost no competing submission can show.

Step 6 — Push to the Space

```
git remote add space https://huggingface.co/spaces/USERNAME/depthwizard
git push space main
```

When prompted, the username is your Hugging Face username and the password is the **write access token** from Step 1 — not your account password.

If your local branch is not named main:

```
git push space HEAD:main
```

Step 7 — Watch the build

- Open the Space page and click the **Logs** tab.
- First build takes 5 to 15 minutes.
- Watch for `Application startup complete`, then your own model-ready log line.
- Status top-right moves from *Building* to *Running*.
- Open the URL and confirm a demo scene renders.

Troubleshooting

Symptom	Cause and fix
Build fails immediately, no logs	YAML frontmatter missing or not at the very top of README.md. It must be the first characters in the file.
Blank page or 502 after a successful build	App not listening on 7860. Check the CMD line and <code>app_port</code> .
Permission denied writing files	Spaces runs non-root. Ensure USER user is set and every COPY uses <code>--chown=user</code> .
ImportError: libGL.so.1	Install <code>libgl1</code> , or switch to <code>opencv-python-headless</code> .
rasterio or GDAL import error	Use a recent rasterio wheel (bundles GDAL). Ensure <code>libexpat1</code> is installed.
Out of memory during build	Install the CPU-only PyTorch wheel. The default pulls CUDA and adds gigabytes.
Container marked unhealthy on first boot	<code>--start-period</code> too short. Model download and load can take several minutes.
Model download fails at startup	Check logs for the actual URL and status code. If the HF Xet CDN returns 503, retry, or fall back to Git LFS.
Space is slow on the first click after a day	Free Spaces sleep after ~48 hours idle. Open the URL 10 minutes before any demo.
Push rejected: file too large	You are pushing weights without LFS. Either gitignore them and download at startup, or configure Git LFS.

Recommended: also deploy a static viewer

Costs nothing extra and cannot break in front of a judge — no cold start, no timeout, no inference failure.

- Export the precomputed demo scenes as static JSON, compressed JPEG textures and GLB meshes.

- Build a viewer-only frontend loading those as static assets. Camera modes, flythrough, click-to-measure, slope overlay, flood slider and provenance badge all still work.
- Disable upload, with a note pointing to the GitHub repository for the full pipeline.
- Deploy to Vercel or Netlify — free tier, no card, connect the GitHub repo and it builds automatically.

Keep textures compressed. A few 1024 x 1024 JPEGs load fast; uncompressed PNGs will make the page slow and undo the advantage.

What to put in the presentation

Artefact	What it shows
Static viewer link (primary)	Interactive 3D with real results, loads in ~2 seconds, cannot fail
Hugging Face Space link	Full live pipeline with upload, for anyone wanting to try their own image
GitHub repository	Source, Dockerfile, evaluation harness, reproducibility
Demo video	Full pipeline end to end — your insurance if anything fails on the day

Record the demo video before you travel. It is the only artefact that survives failure of everything else.

Final checklist

- Everything merged to main and pushed to GitHub
- Repository public so judges can see it
- README shows a one-command run, screenshots and the honest results table
- requirements.txt pinned; random seeds fixed
- STATUS.md records what works, what is untested, and known limitations
- Space opened and warmed on the morning of the demo
- Demo video recorded and hosted at a stable link